Presencia

CONTROL DE ASISTENCIA

DOCUMENTACIÓN TÉCNICA · VERSIÓN 1.0

Sistema de registro de asistencia

Base de datos normalizada, arquitectura, reglas de negocio y pantallas

Elaborado por

Grupo GPT-6 AURA

Diplomado · 13 de octubre de 2026

FastAPI · Next.js 16 · PostgreSQL 16

Contenido

1 Introducción 4

1.1 Cumplimiento de los requisitos 4

2 Arquitectura 5

2.1 Tecnologías 5

2.2 Decisiones relevantes 5

3 Base de datos 6

3.1 Modelo entidad-relación 7

3.2 Normalización 8

3.3 Decisiones de diseño 8

3.4 Tipos enumerados 8

3.5 Diccionario de datos 9

4 Reglas de negocio 13

4.1 Registro de asistencia 14

4.2 Importación desde Excel 15

5 Pantallas 16

5.1 Inicio de sesión 16

5.2 Animación de bienvenida · «App hecha por el grupo» 17

5.3 Animación de bienvenida · «GPT-6 AURA» 18

5.4 Animación de bienvenida · cenizas 18

5.5 Panel del administrador 19

5.6 Programas 20

5.7 Estudiantes 21

5.8 Ficha del estudiante (modal) 22

5.9 Sesiones y asistencia 23

5.10 Control de asistencia de una sesión 24

5.11 Importaciones 25

5.12 Modal de importación en dos pasos 26

5.13 Modo oscuro 27

5.14 Estudiante · registrar hoy 28

5.15 Estudiante · historial 29

5.16 Vista móvil . . . 30
5.17 Menú móvil en modal . . . 32
6 Referencia de la API . . . 33
7 Modelos de importación Excel . . . 34
7.1 Estudiantes · modelo_estudiantes.xlsx . . . 34
7.2 Horarios · modelo_horarios.xlsx . . . 34
8 Instalación y ejecución . . . 35
8.1 Con Docker (recomendado) . . . 35
8.2 Desarrollo local . . . 35
8.3 Variables de entorno principales . . . 35
8.4 Calidad . . . 35

1 Introducción

Presencia es un sistema web para registrar y controlar la asistencia de los estudiantes en uno o varios programas académicos. Los administradores importan estudiantes, horarios y fechas desde Excel, revisan la asistencia de todos y la corrigen cuando hace falta. Cada estudiante registra su propia asistencia del día con un solo toque.

Este documento describe la base de datos normalizada, la arquitectura, las reglas de negocio y cada una de las pantallas del sistema.

1.1 Cumplimiento de los requisitos

Requisito del docente	Cómo lo resuelve Presencia
Registro de asistencia funcional	Aplicación completa (FastAPI + Next.js + PostgreSQL) que se levanta con `docker compose up`. 25 pruebas automatizadas cubren las reglas.
Diagramas	Diagramas en PlantUML: entidad-relación, arquitectura, casos de uso y dos diagramas de secuencia (carpeta `docs/diagrams`).
GitHub	Repositorio privado `github.com/Aliaga23/diplo-asistencia` con integración continua.
Tipos de usuario (admin, estudiante)	Columna `users.role` con enum nativo. El backend valida el rol en cada endpoint y el frontend redirige por rol.
Importar datos desde Excel	Importación de estudiantes con modelo descargable, validación fila por fila e inscripción automática al programa.
Importar horarios y fechas desde Excel	Importación de sesiones con varios formatos de fecha y hora; si la sesión ya existe, se actualiza.
El admin ve a todos y modifica su asistencia	Pantallas de Estudiantes, ficha del estudiante y Control de asistencia por sesión, con edición de cualquier fecha y notas.
Cada estudiante registra su asistencia	Botón «Registrar mi asistencia»; el estado (presente o tarde) lo calcula el servidor según la hora de inicio y la tolerancia.
El estudiante solo modifica su asistencia del día	El servidor compara la fecha de la sesión con la fecha actual en America/La_Paz. Registrar o anular otra fecha responde 403.

> **Acceso de demostración**
> Todas las cuentas usan la contraseña común `admin1234!`. Administrador: registro `218166141`. Estudiante de ejemplo: registro `219034521` (inscrito en dos programas).

2 Arquitectura

El sistema sigue una arquitectura de tres capas con un **backend-for-frontend** (BFF) en Next.js. El navegador nunca recibe el token: el servidor de Next.js lo guarda en una cookie httpOnly y lo adjunta en cada llamada a la API.

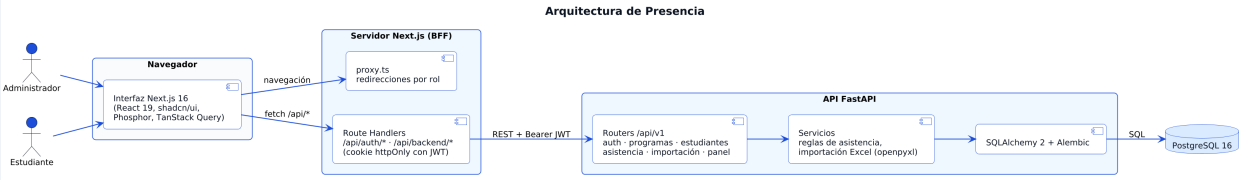


Figura 1. Arquitectura de Presencia.

2.1 Tecnologías

Capa	Tecnología	Motivo
Interfaz	Next.js 16 (App Router, Cache Components), React 19, TypeScript	Renderizado híbrido, rutas por carpeta y tipado estricto.
Componentes	shadcn/ui (CLI), Tailwind CSS v4, Phosphor Icons	Componentes accesibles y personalizables; íconos Phosphor en lugar de Lucide.
Datos en cliente	TanStack Query, React Hook Form, Zod	Caché, reintentos y validación de formularios.
API	FastAPI, Pydantic v2, PyJWT, pwdlib (Argon2)	API REST tipada con documentación OpenAPI automática.
Persistencia	PostgreSQL 16, SQLAlchemy 2.0, Alembic	Integridad referencial, enums nativos y migraciones versionadas.
Excel	openpyxl	Lectura de importaciones y generación de modelos y reportes.
Calidad	pytest, Ruff, ESLint, GitHub Actions	Pruebas y análisis en cada cambio.
Despliegue	Docker Compose (db, backend, frontend)	Un solo comando para levantar todo.

2.2 Decisiones relevantes

- **Hora oficial en el servidor.** «Hoy» se calcula siempre en el backend con la zona America/La_Paz; nunca se confía en el reloj del navegador.
- **Sesión segura.** JWT firmado (HS256) en cookie httpOnly y SameSite=Lax. El proxy de Next.js solo hace redirecciones optimistas; la autorización real está en la API.
- **Contraseñas.** Hash Argon2. Todas las cuentas nuevas reciben la contraseña común configurable `DEFAULT_PASSWORD` y pueden cambiarla desde su menú.
- **Multiprograma.** Toda la información (estudiantes, sesiones, indicadores, importaciones) se filtra por el programa activo.

3 Base de datos

La base de datos está en PostgreSQL 16 y se crea con migraciones de Alembic. Usa claves primarias UUID, enums nativos, claves foráneas con reglas de borrado explícitas y restricciones de unicidad y verificación que protegen las reglas de negocio desde el propio motor.

Tabla	Propósito	Relación principal
`users`	Cuentas de acceso. Concentra solo datos de autenticación y rol.	1:1 con students
`students`	Perfil académico del estudiante, especialización 1:1 de users.	N:M con courses mediante enrollments
`courses`	Programas académicos (cursos o diplomados) con control de asistencia propio.	1:N con class_sessions
`enrollments`	Tabla asociativa N:M entre estudiantes y programas.	students ↔ courses
`class_sessions`	Horarios y fechas de clase de cada programa.	1:N con attendance_records
`attendance_records`	Un registro de asistencia por estudiante y sesión.	class_sessions ↔ students
`import_batches`	Auditoría de cada archivo Excel importado.	N:1 con courses y users

3.1 Modelo entidad-relación

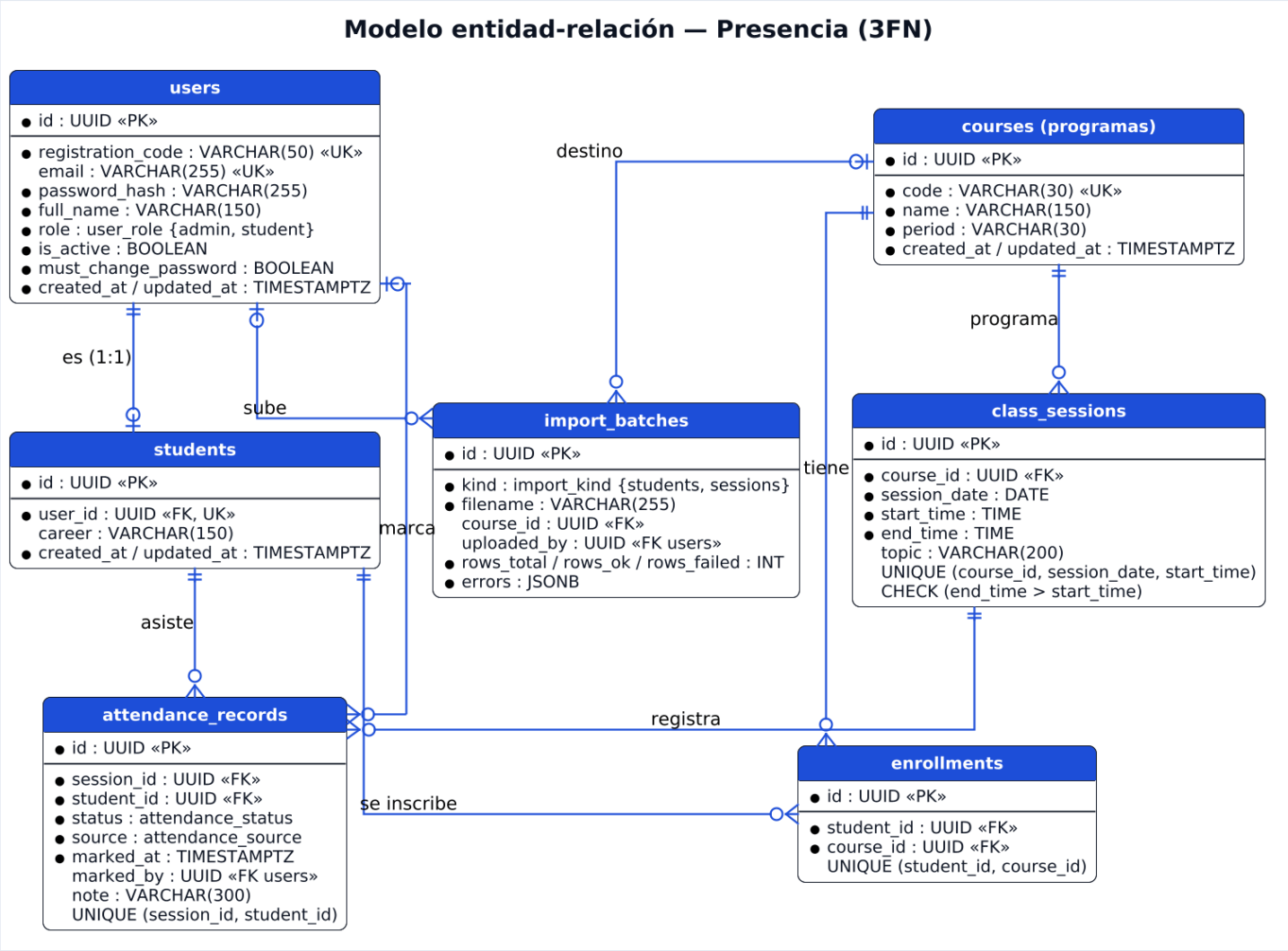


Figura 2. Modelo entidad-relación (PlantUML). PK: clave primaria · FK: clave foránea · UK: clave única.

3.2 Normalización

El modelo cumple la **tercera forma normal (3FN)** y, en la práctica, la forma normal de Boyce-Codd: cada determinante es una clave candidata.

Primera forma normal (1FN). Todos los atributos son atómicos y no hay grupos repetidos. Un estudiante inscrito en dos programas genera dos filas en `enrollments`, nunca una lista dentro de una columna. Cada sesión es una fila propia en `class_sessions`.

Segunda forma normal (2FN). No hay dependencias parciales de una clave compuesta. En `attendance_records` la combinación (session_id, student_id) es única y todos los atributos (estado, origen, hora, nota) dependen de esa combinación completa: describen la asistencia de *ese* estudiante en *esa* sesión.

Tercera forma normal (3FN). No hay dependencias transitivas. El nombre del programa vive solo en `courses` y no se copia en las sesiones; el nombre del estudiante vive solo en `users` y no se repite en los registros de asistencia. El número de registro se guarda una única vez, en `users.registration_code`, porque identifica a cualquier usuario al ingresar.

3.3 Decisiones de diseño

- **Autenticación separada del perfil académico.** `users` guarda credenciales y rol; `students` guarda los datos académicos. Un administrador no tiene filas vacías de datos de estudiante.
- **Las faltas no se almacenan por defecto.** Si un estudiante no tiene registro en una sesión ya dictada, se calcula como falta. Así no se guardan datos derivados ni se crean miles de filas vacías; el administrador puede fijar una falta explícita con nota.
- **Sesión «dictada».** Una sesión cuenta para los porcentajes solo cuando su fecha pasó o terminó su horario de hoy; así nadie aparece con falta antes de que acabe la clase.
- **Trazabilidad.** Cada registro guarda `source`, `marked_by` y `marked_at`; cada importación queda en `import_batches` con el detalle de errores.
- **Integridad en el motor.** `UNIQUE(session_id, student_id)` impide registros duplicados, `UNIQUE(course_id, session_date, start_time)` impide sesiones repetidas y `CHECK(end_time > start_time)` impide horarios inválidos.

3.4 Tipos enumerados

Enum	Valores	Significado
`user_role`	`admin, student`	Rol de la cuenta.
`attendance_status`	`present, late, absent, excused`	Presente, tarde, falta y justificado.
`attendance_source`	`self, admin, import`	Quién originó el registro: el estudiante, un administrador o una importación.
`import_kind`	`students, sessions`	Tipo de archivo importado.

3.5 Diccionario de datos

Generado automáticamente a partir del modelo SQLAlchemy, por lo que coincide con la migración aplicada en PostgreSQL.

`courses` — Programas académicos (cursos o diplomados) con control de asistencia propio.

Columna	Tipo	Restricciones	Descripción
`code`	`varchar(30)`	UNIQUE NOT NULL	Código corto y único del programa (p. ej. DIP-FS-2026).
`name`	`varchar(150)`	NOT NULL	Nombre del programa.
`period`	`varchar(30)`	NOT NULL	Gestión o periodo académico.
`id`	`uuid`	PK NOT NULL	Identificador del programa.
`created_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de creación (auditoría).
`updated_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de la última modificación (auditoría).

`users` — Cuentas de acceso. Concentra solo datos de autenticación y rol.

Columna	Tipo	Restricciones	Descripción
`registration_code`	`varchar(50)`	UNIQUE NOT NULL	Número de registro institucional. Es el usuario de ingreso de todos.
`email`	`varchar(255)`	UNIQUE NULL	Correo institucional (opcional, único si existe).
`password_hash`	`varchar(255)`	NOT NULL	Hash Argon2 de la contraseña. Nunca se guarda la contraseña en texto.
`full_name`	`varchar(150)`	NOT NULL	Nombre completo.
`role`	`user_role`	NOT NULL	Rol del usuario: admin o student.
`is_active`	`boolean`	NOT NULL	Permite desactivar una cuenta sin borrar su historial.
`must_change_password`	`boolean`	NOT NULL	Obliga a cambiar la contraseña en el próximo ingreso.
`id`	`uuid`	PK NOT NULL	Identificador único (UUID v4).
`created_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de creación (auditoría).
`updated_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de la última modificación (auditoría).

Índices: `registration_code`

`class_sessions` — Horarios y fechas de clase de cada programa.

Columna	Tipo	Restricciones	Descripción
`course_id`	`uuid`	FK → courses.id · ON DELETE CASCADE NOT NULL	Programa al que pertenece la sesión.
`session_date`	`date`	NOT NULL	Fecha de la clase.
`start_time`	`time without time zone`	NOT NULL	Hora de inicio (base del cálculo de tardanza).
`end_time`	`time without time zone`	NOT NULL	Hora de fin; debe ser posterior al inicio.
`topic`	`varchar(200)`	NULL	Tema o contenido de la sesión.
`id`	`uuid`	PK NOT NULL	Identificador de la sesión.
`created_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de creación (auditoría).
`updated_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de la última modificación (auditoría).

Unicidad compuesta: `(course_id, session_date, start_time)` · Verificación: `end_time > start_time` · Índices: `session_date`, `course_id`

`import_batches` — Auditoría de cada archivo Excel importado.

Columna	Tipo	Restricciones	Descripción
`kind`	`import_kind`	NOT NULL	Tipo de importación: students o sessions.
`filename`	`varchar(255)`	NOT NULL	Nombre del archivo Excel procesado.
`course_id`	`uuid`	FK → courses.id · ON DELETE SET NULL NULL	Programa destino de la importación.
`uploaded_by`	`uuid`	FK → users.id · ON DELETE SET NULL NULL	Administrador que subió el archivo.
`rows_total`	`integer`	NOT NULL	Filas leídas del archivo.
`rows_ok`	`integer`	NOT NULL	Filas procesadas correctamente.
`rows_failed`	`integer`	NOT NULL	Filas rechazadas.
`errors`	`jsonb`	NOT NULL	Detalle de errores por fila en formato JSON.
`id`	`uuid`	PK NOT NULL	Identificador del lote de importación.
`created_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de creación (auditoría).
`updated_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de la última modificación (auditoría).

`students` — Perfil académico del estudiante, especialización 1:1 de users.

Columna	Tipo	Restricciones	Descripción
`user_id`	`uuid`	FK → users.id · ON DELETE CASCADE UNIQUE NOT NULL	Cuenta asociada. Relación 1:1 con users.
`career`	`varchar(150)`	NULL	Carrera o especialidad del estudiante.
`id`	`uuid`	PK NOT NULL	Identificador del perfil académico.
`created_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de creación (auditoría).
`updated_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de la última modificación (auditoría).

`attendance_records` — Un registro de asistencia por estudiante y sesión.

Columna	Tipo	Restricciones	Descripción
`session_id`	`uuid`	FK → class_sessions.id · ON DELETE CASCADE NOT NULL	Sesión de clase.
`student_id`	`uuid`	FK → students.id · ON DELETE CASCADE NOT NULL	Estudiante.
`status`	`attendance_status`	NOT NULL	Estado: present, late, absent o excused.
`source`	`attendance_source`	NOT NULL	Origen del registro: self (estudiante), admin o import.
`marked_at`	`timestamp with time zone`	NOT NULL	Momento exacto del registro (con zona horaria).
`marked_by`	`uuid`	FK → users.id · ON DELETE SET NULL NULL	Usuario que hizo el registro (estudiante o administrador).
`note`	`varchar(300)`	NULL	Observación opcional (p. ej. motivo de una justificación).
`id`	`uuid`	PK NOT NULL	Identificador del registro.
`created_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de creación (auditoría).
`updated_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de la última modificación (auditoría).

Unicidad compuesta: `(session_id, student_id)` · Índices: `session_id`, `student_id`

`enrollments` — Tabla asociativa N:M entre estudiantes y programas.

Columna	Tipo	Restricciones	Descripción
`student_id`	`uuid`	FK → students.id · ON DELETE CASCADE NOT NULL	Estudiante inscrito.
`course_id`	`uuid`	FK → courses.id · ON DELETE CASCADE NOT NULL	Programa en el que se inscribe.
`id`	`uuid`	PK NOT NULL	Identificador de la inscripción.
`created_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de creación (auditoría).
`updated_at`	`timestamp with time zone`	NOT NULL DEFAULT now()	Fecha de la última modificación (auditoría).

Unicidad compuesta: `(student_id, course_id)` · Índices: `course_id`, `student_id`

4 Reglas de negocio

- **Solo el día de hoy.** El estudiante solo puede registrar o anular su asistencia en sesiones cuya fecha es la fecha actual en America/La_Paz. Otras fechas responden 403.
- **Presente o tarde.** Si registra antes de la hora de inicio más la tolerancia (15 minutos, configurable) queda «presente»; después, «tarde».
- **Registro idempotente.** Volver a pulsar el botón no cambia el registro, así nadie pasa de presente a tarde por error.
- **Anulación.** El estudiante puede anular su propio registro durante el día. Un registro hecho o confirmado por el administrador no puede ser modificado por el estudiante.
- **Administrador.** Puede asignar cualquier estado (presente, tarde, falta, justificado) en cualquier fecha y agregar una nota.
- **Inscripción.** El estudiante solo ve y registra sesiones de los programas en los que está inscrito.
- **Importación idempotente.** Reimportar el mismo archivo no duplica estudiantes, inscripciones ni sesiones.

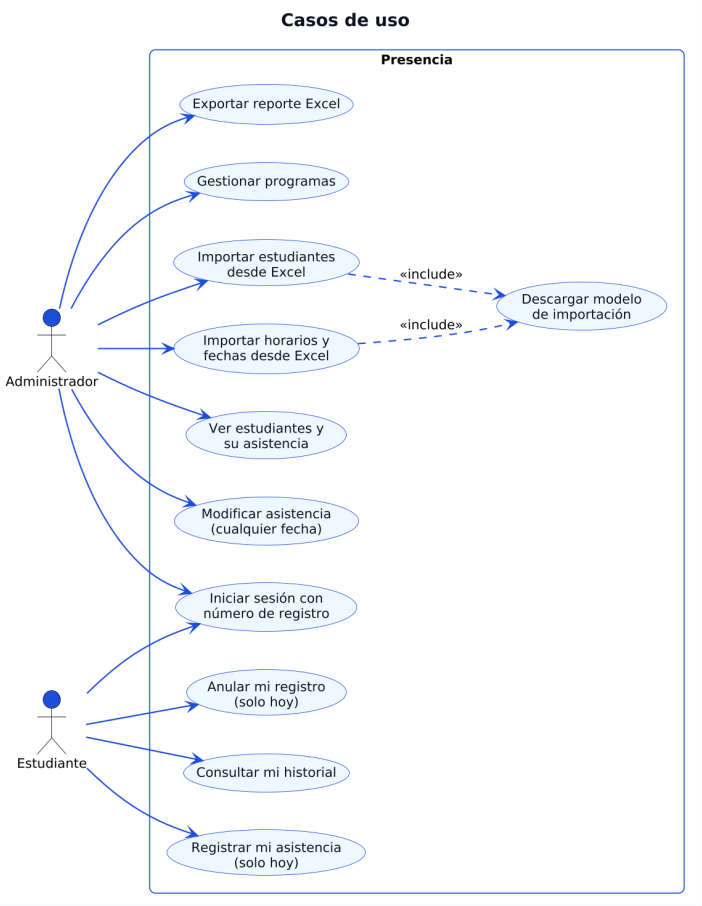


Figura 3. Casos de uso.

4.1 Registro de asistencia

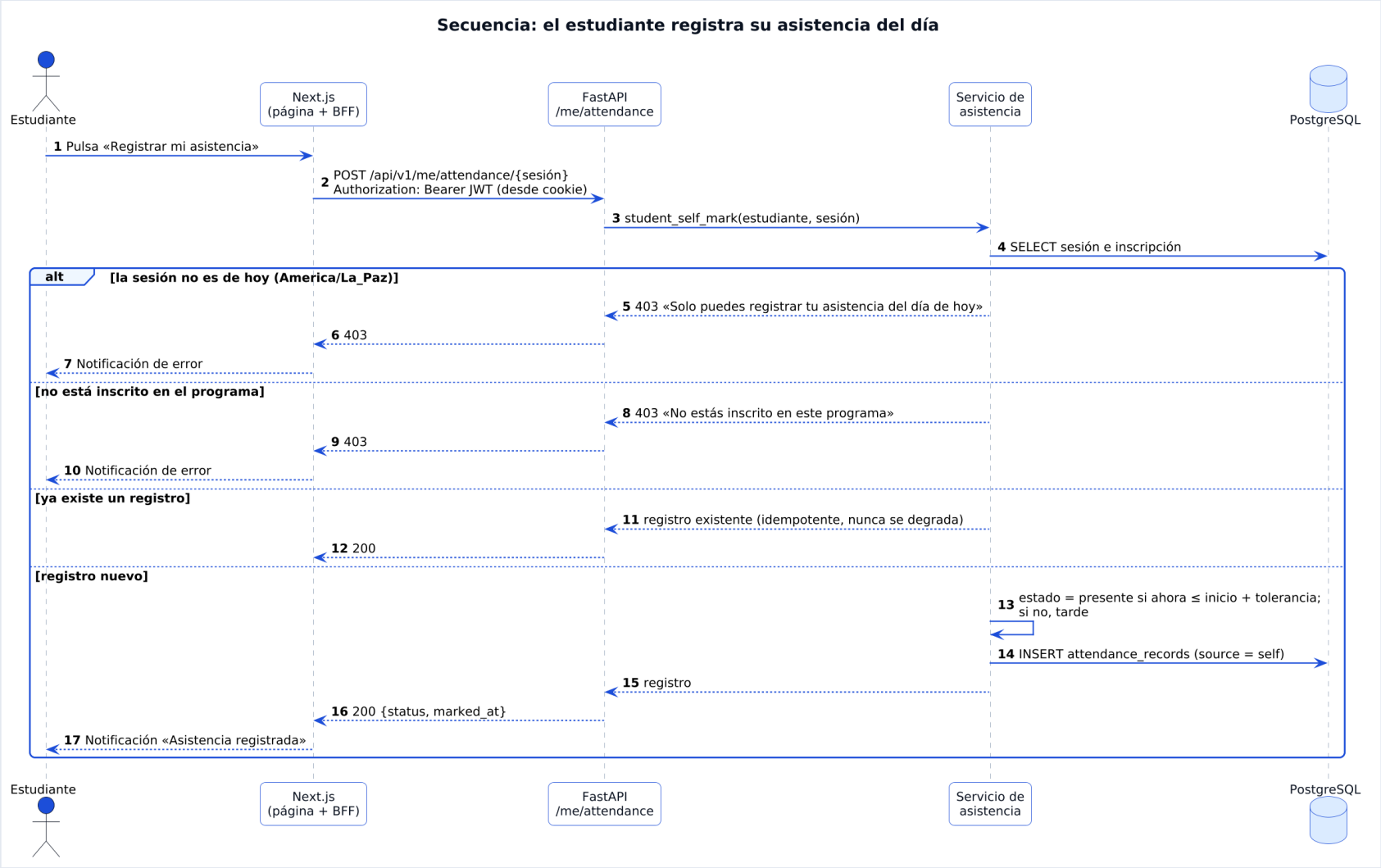


Figura 4. Secuencia del registro de asistencia del estudiante.

4.2 Importación desde Excel

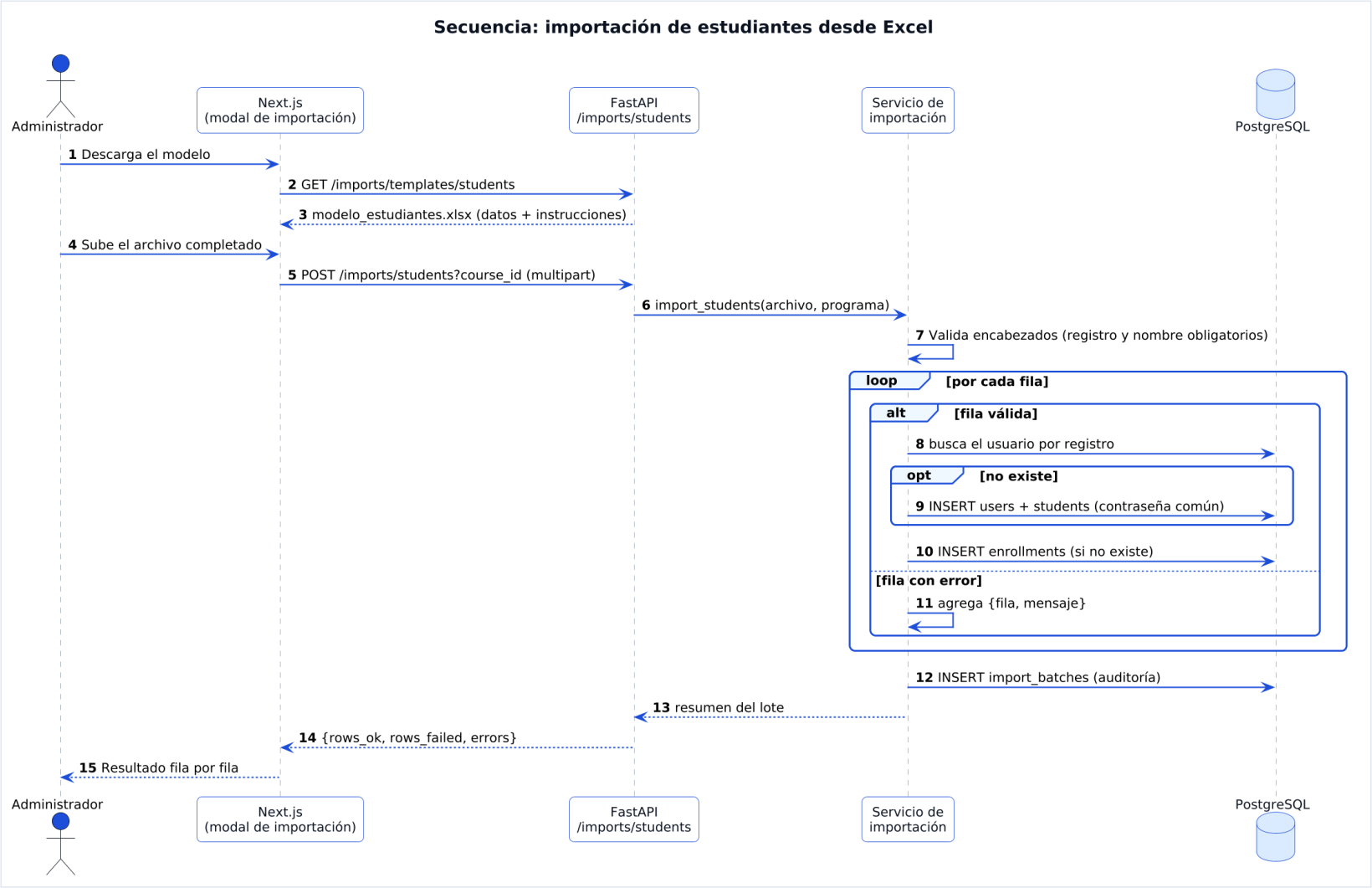


Figura 5. Secuencia de la importación de estudiantes.

5 Pantallas

La interfaz usa una paleta azul, negro y blanco, tipografía Geist, íconos Phosphor y componentes shadcn/ui. Las ventanas emergentes son modales propios, las notificaciones aparecen abajo a la derecha y todo funciona en tema claro, oscuro y en celulares.

5.1 Inicio de sesión

Pantalla dividida: a la izquierda la propuesta de valor del sistema y a la derecha el formulario. El ingreso es con **número de registro** y contraseña; la sesión se guarda en una cookie httpOnly que el navegador no puede leer.

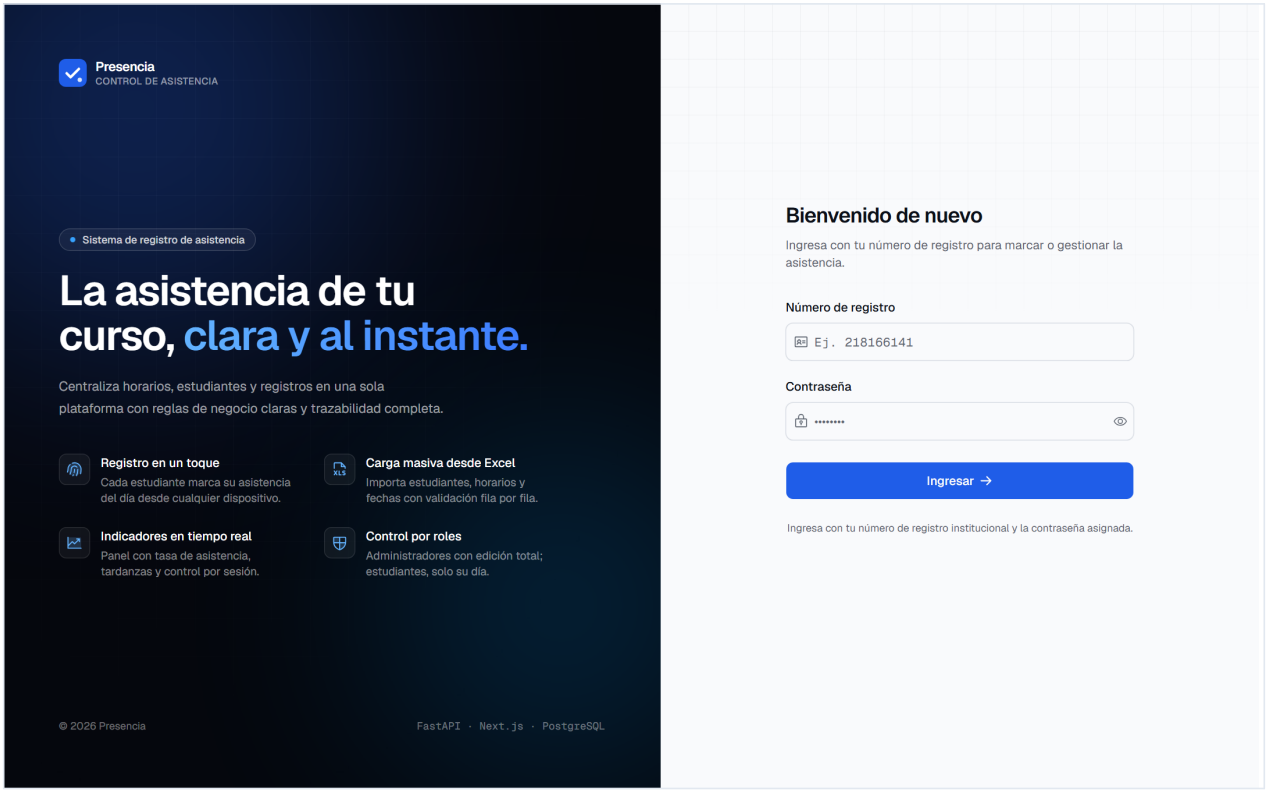


Figura 6. Inicio de sesión.

5.2 Animación de bienvenida · «App hecha por el grupo»

Tras un ingreso correcto aparece una pantalla negra con destellos. Las partículas se ensamblan en el texto «App hecha por el grupo». Se puede saltar con el botón, Escape o Enter, y respeta la preferencia de movimiento reducido del sistema operativo.



Figura 7. Animación de bienvenida · «App hecha por el grupo».

5.3 Animación de bienvenida · «GPT-6 AURA»

Segundo ensamblaje con el nombre del grupo en blanco y azul, con un barrido de luz.



Figura 8. Animación de bienvenida · «GPT-6 AURA».

5.4 Animación de bienvenida · cenizas

Todo se desintegra en cenizas de izquierda a derecha y revela el panel de destino, que se cargó por detrás durante la animación.

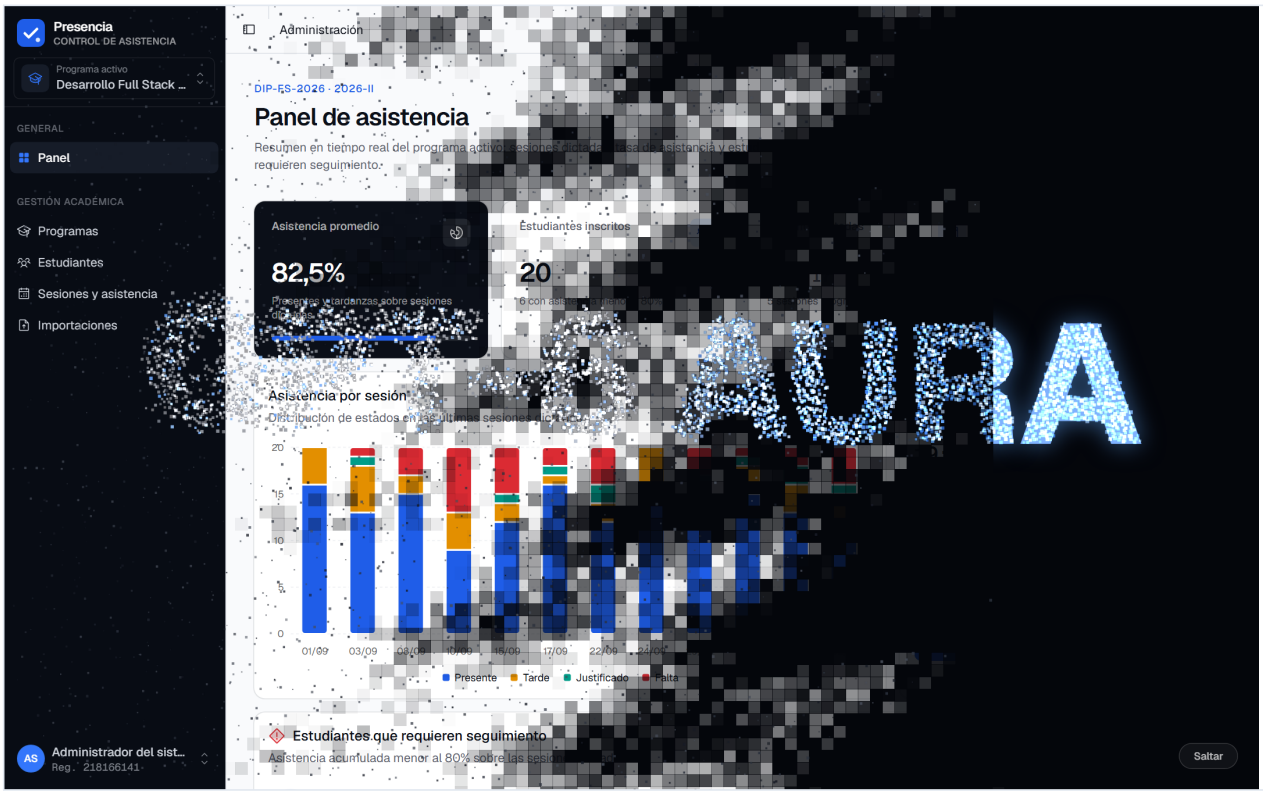


Figura 9. Animación de bienvenida · cenizas.

5.5 Panel del administrador

Indicadores del programa activo: asistencia promedio, estudiantes inscritos, sesiones dictadas y registros del día. Incluye el gráfico de barras apiladas por sesión, el control de la sesión de hoy y la lista de estudiantes con asistencia menor al 80%.

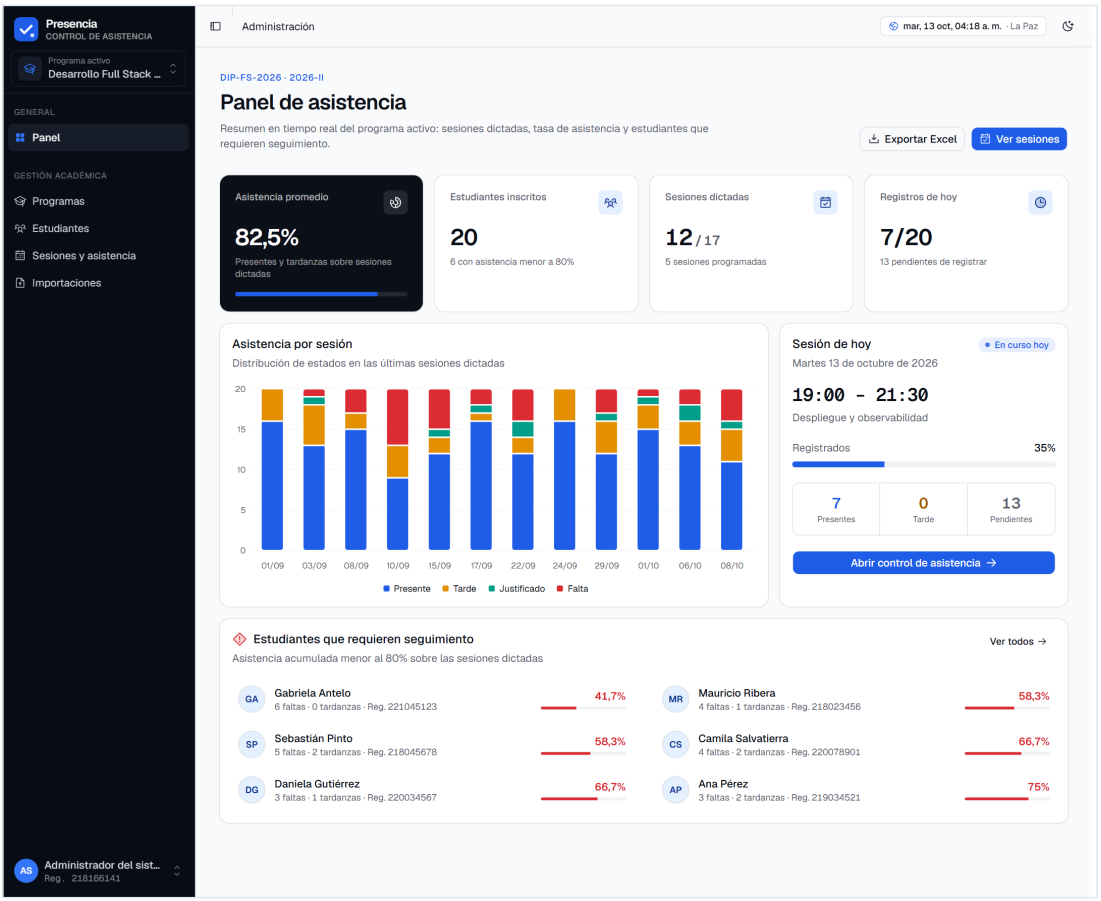


Figura 10. Panel del administrador.

5.6 Programas

El control de asistencia funciona para cualquier número de programas. Cada tarjeta muestra estudiantes, sesiones y asistencia promedio; desde aquí se crea, edita o activa un programa.

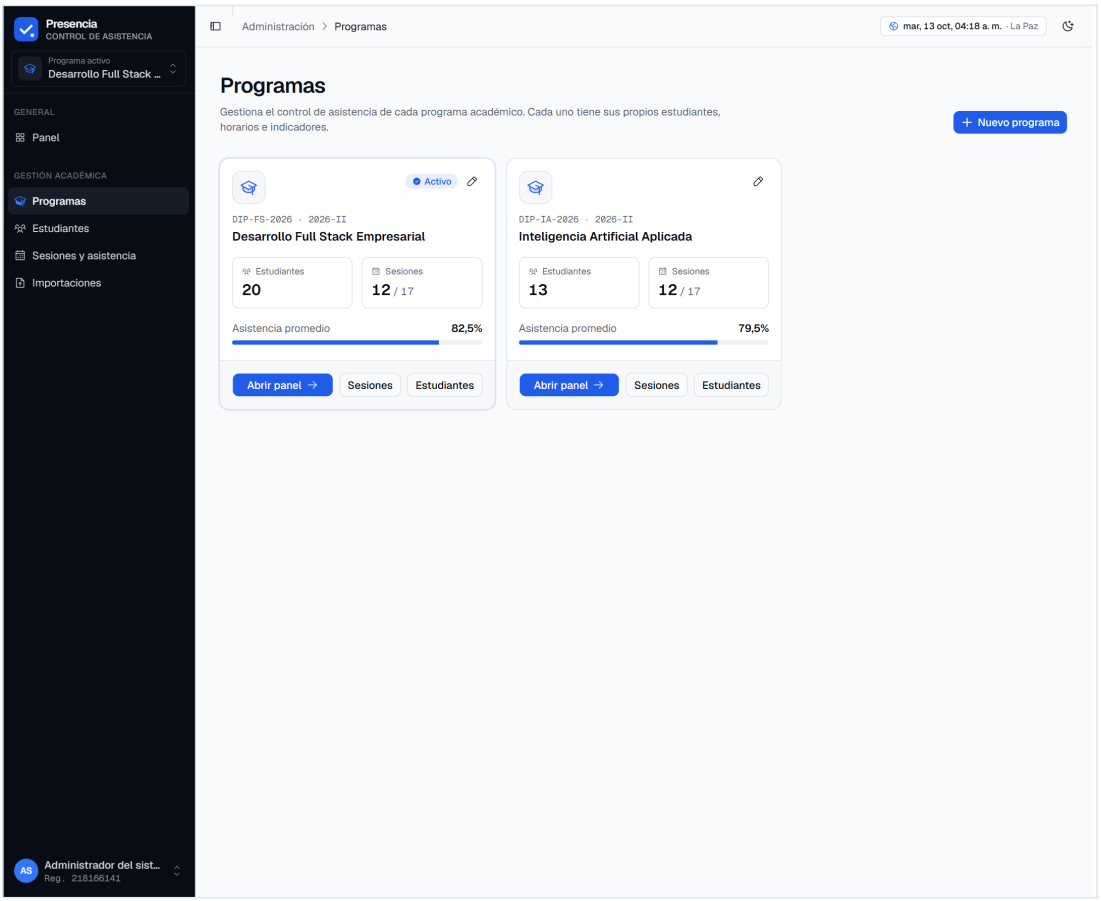


Figura 11. Programas.

5.7 Estudiantes

Listado del programa activo con búsqueda por nombre, registro o correo, filtro de riesgo y asistencia acumulada. Se puede importar desde Excel o registrar un estudiante manualmente.

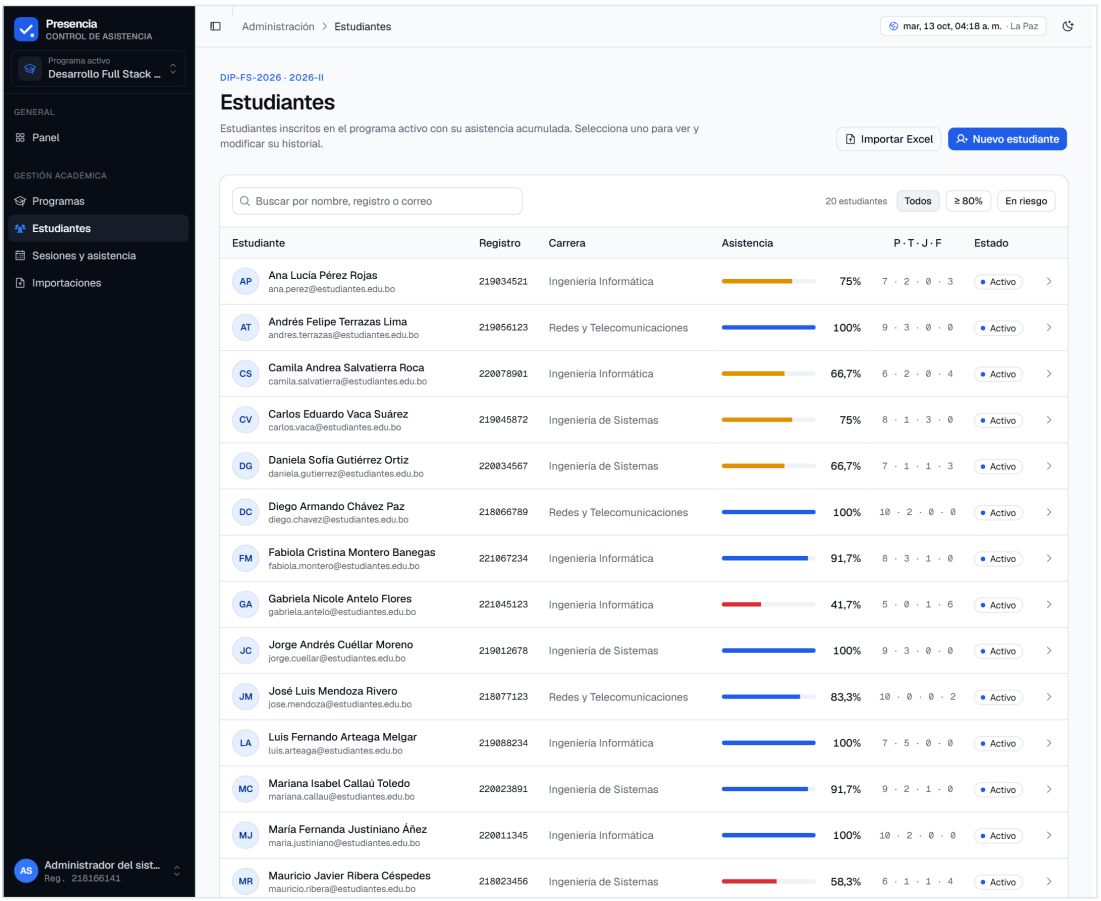


Figura 12. Estudiantes.

5.8 Ficha del estudiante (modal)

Resumen de asistencia, historial del programa con pestañas Dictadas/Programadas y edición del estado de cualquier fecha. Permite restablecer la contraseña y activar o desactivar la cuenta. Todas las ventanas emergentes son modales propios, sin paneles laterales.

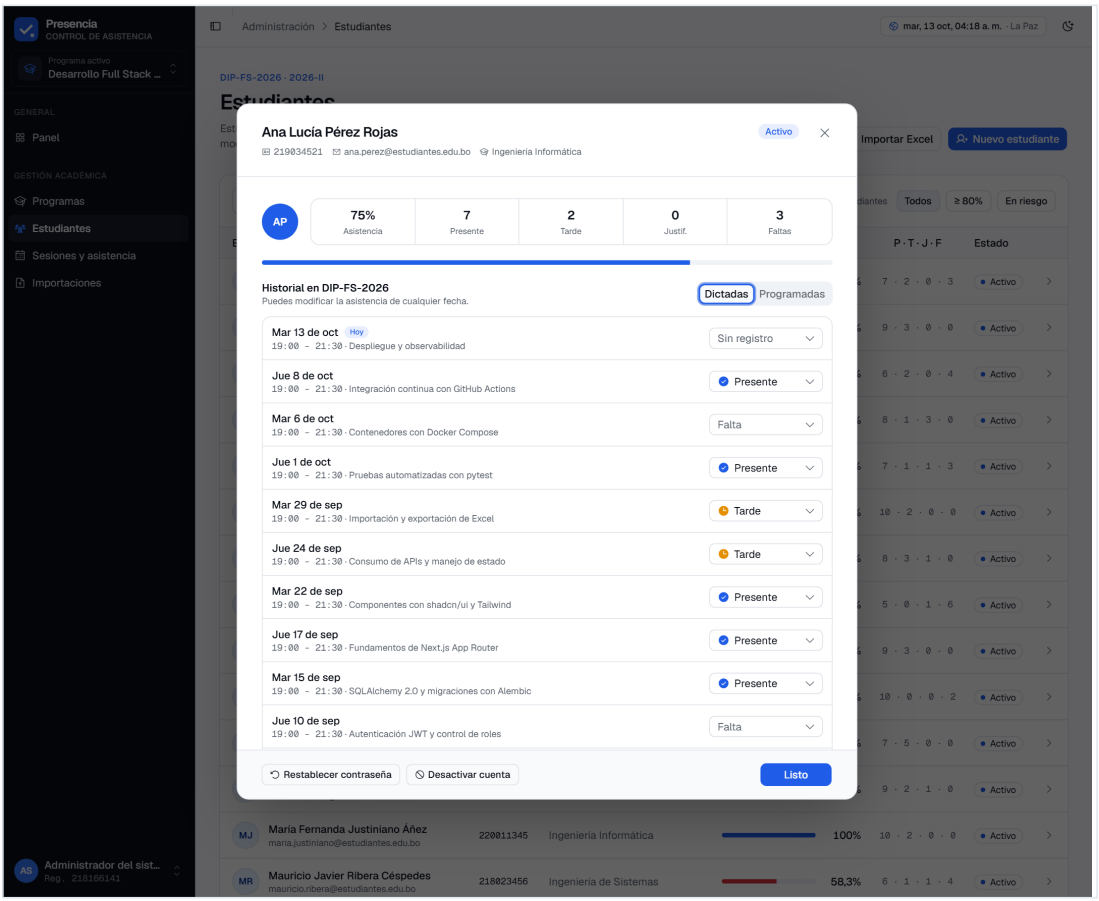


Figura 13. Ficha del estudiante (modal).

5.9 Sesiones y asistencia

Agenda de sesiones con pestañas Próximas/Dictadas, distribución de la asistencia por sesión y acceso al control. Se pueden importar horarios desde Excel o crear sesiones a mano.

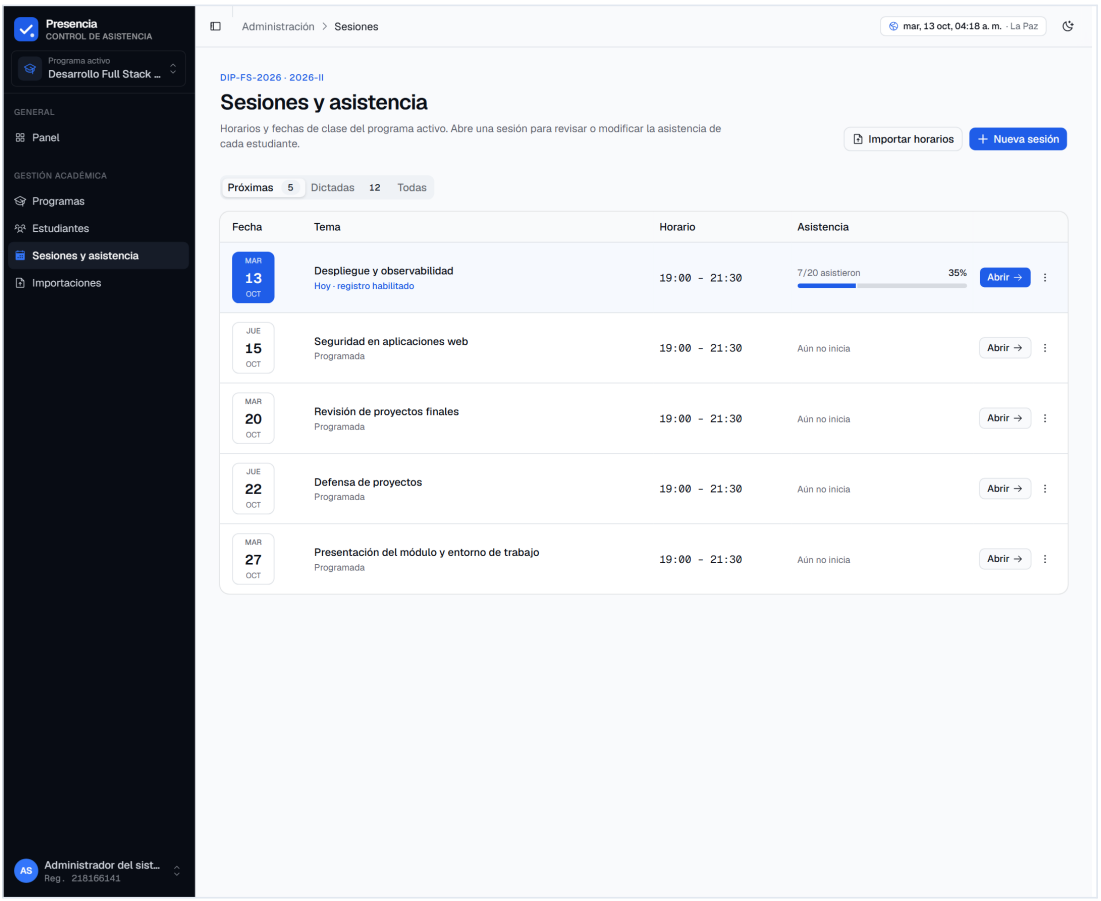


Figura 14. Sesiones y asistencia.

5.10 Control de asistencia de una sesión

Matriz de estudiantes de la sesión con hora de marcado, origen del registro, nota y estado editable. Los contadores superiores funcionan también como filtros.

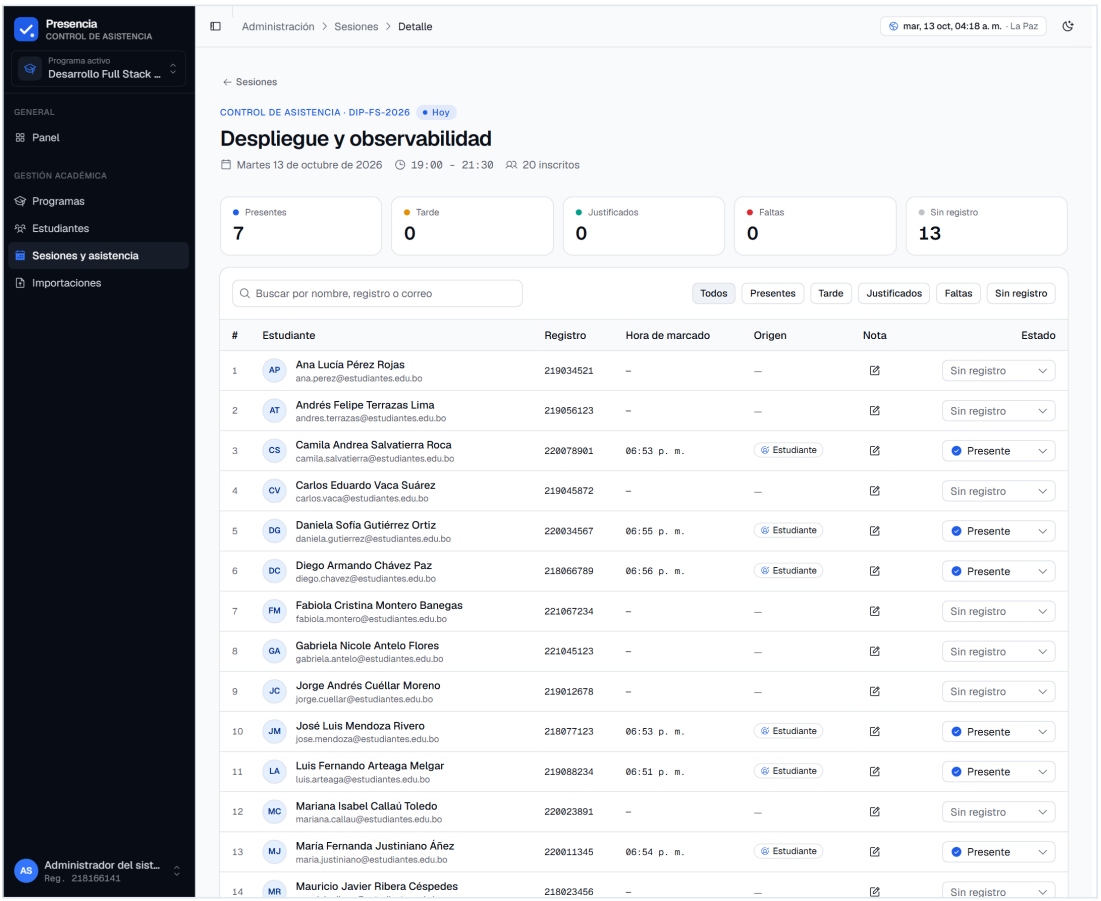


Figura 15. Control de asistencia de una sesión.

5.11 Importaciones

Acceso a los modelos de Excel y a la importación. El historial de lotes queda como registro de auditoría con filas correctas y con error.

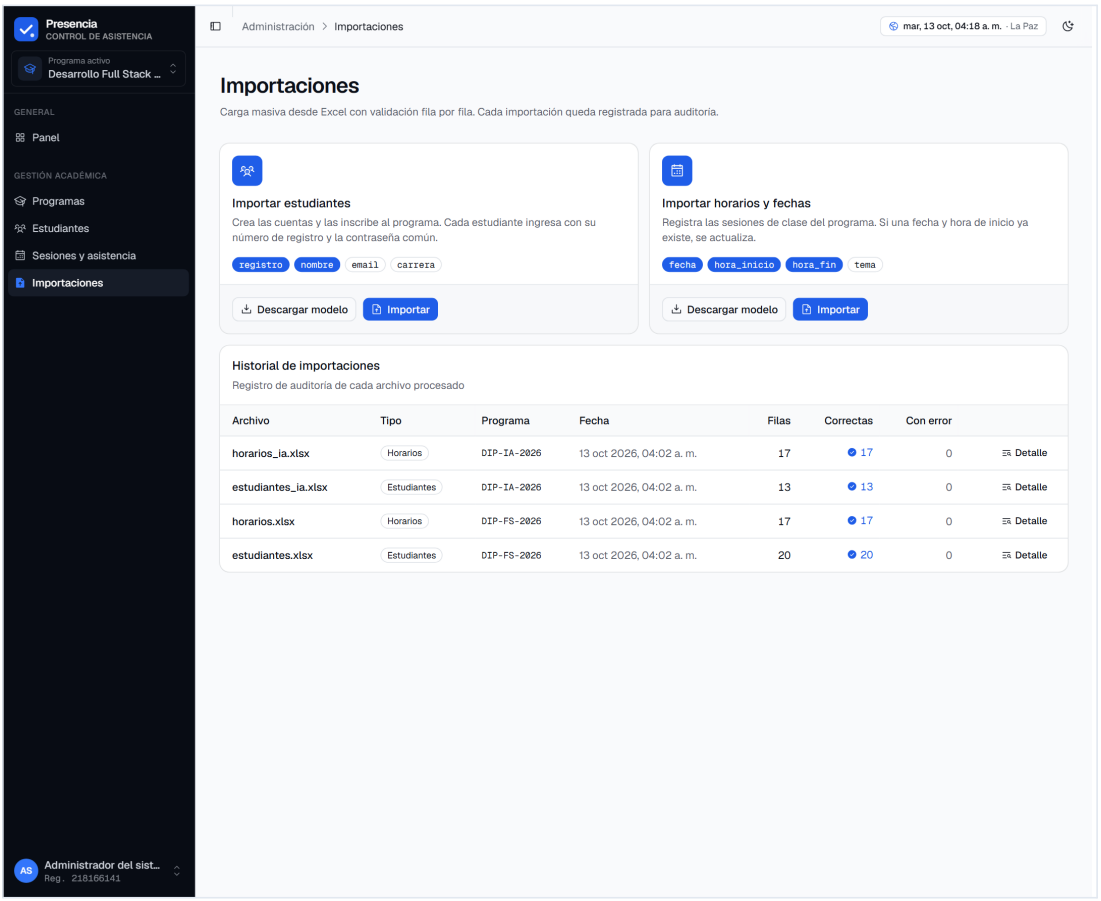


Figura 16. Importaciones.

5.12 Modal de importación en dos pasos

Paso 1: descargar el modelo (incluye ejemplos e instrucciones). Paso 2: subir el archivo completado. El resultado muestra el detalle fila por fila.

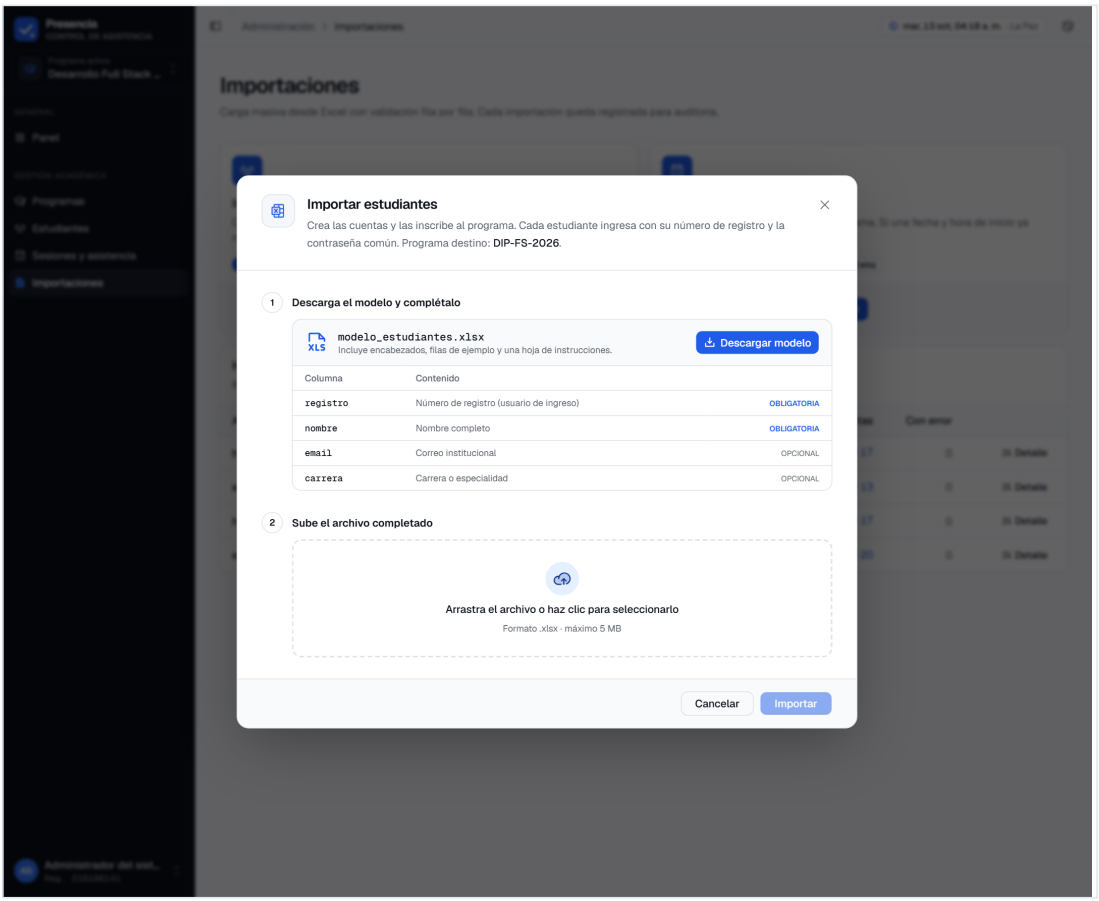


Figura 17. Modal de importación en dos pasos.

5.13 Modo oscuro

Toda la interfaz tiene tema claro y oscuro con la misma paleta azul, negro y blanco.

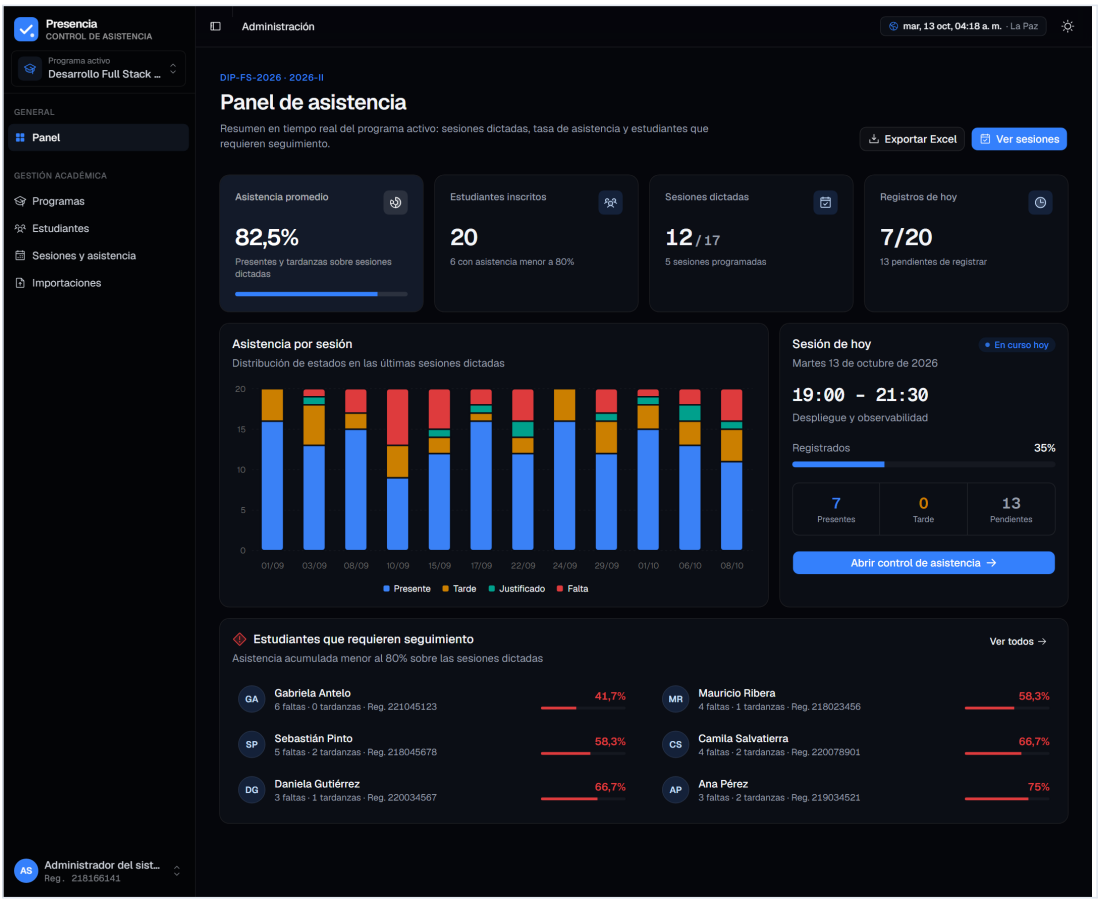


Figura 18. Modo oscuro.

5.14 Estudiante · registrar hoy

Tarjeta por cada clase del día (de todos sus programas) con un único botón para registrar. Indica hasta qué hora cuenta como presente. El estudiante puede anular su propio registro solo durante el día; si el administrador lo confirmó, queda bloqueado.

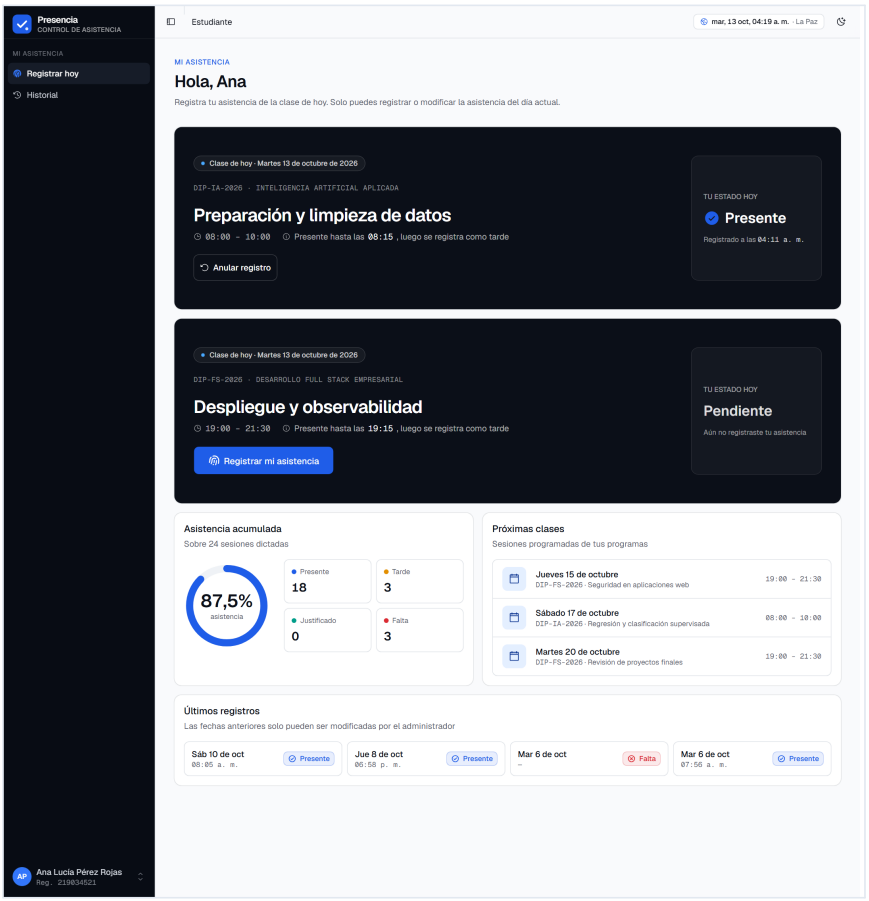


Figura 19. Estudiante · registrar hoy.

5.15 Estudiante · historial

Historial por programa con estado, hora y origen de cada registro. Las fechas anteriores aparecen bloqueadas: solo el administrador puede modificarlas.

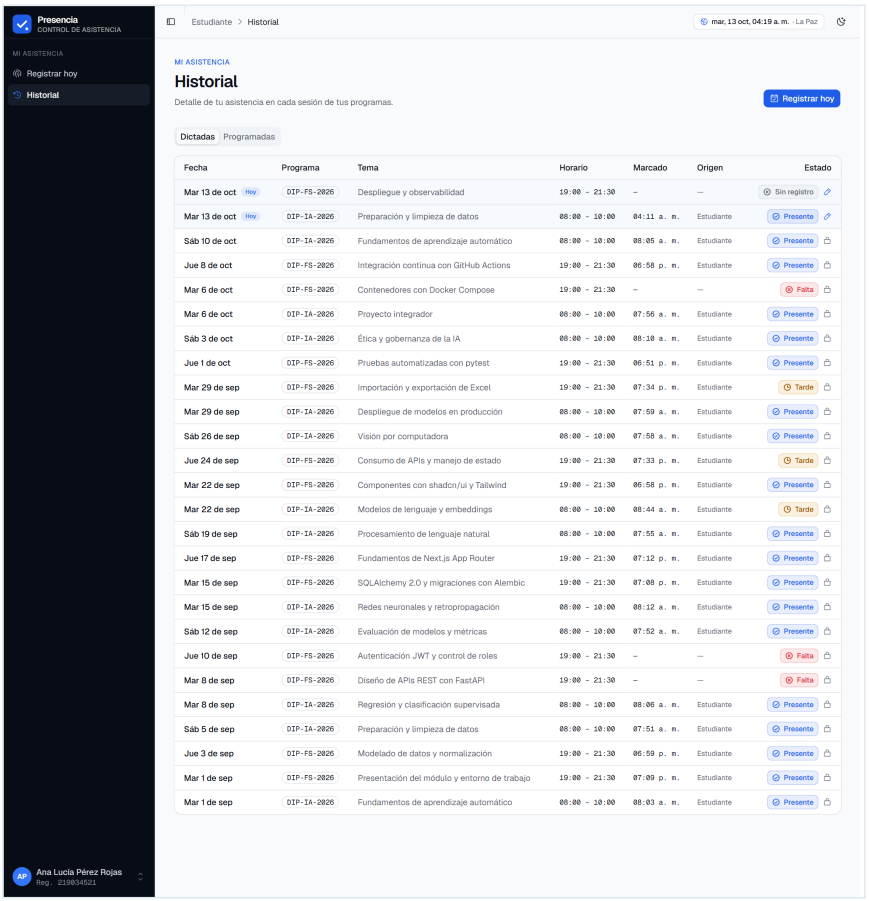


Figura 20. Estudiante · historial.

5.16 Vista móvil

Diseño adaptable: el estudiante registra su asistencia desde el celular con un toque.



Figura 21. Vista móvil.

5.17 Menú móvil en modal

En pantallas pequeñas la navegación se abre en un modal centrado, nunca desde un lateral.

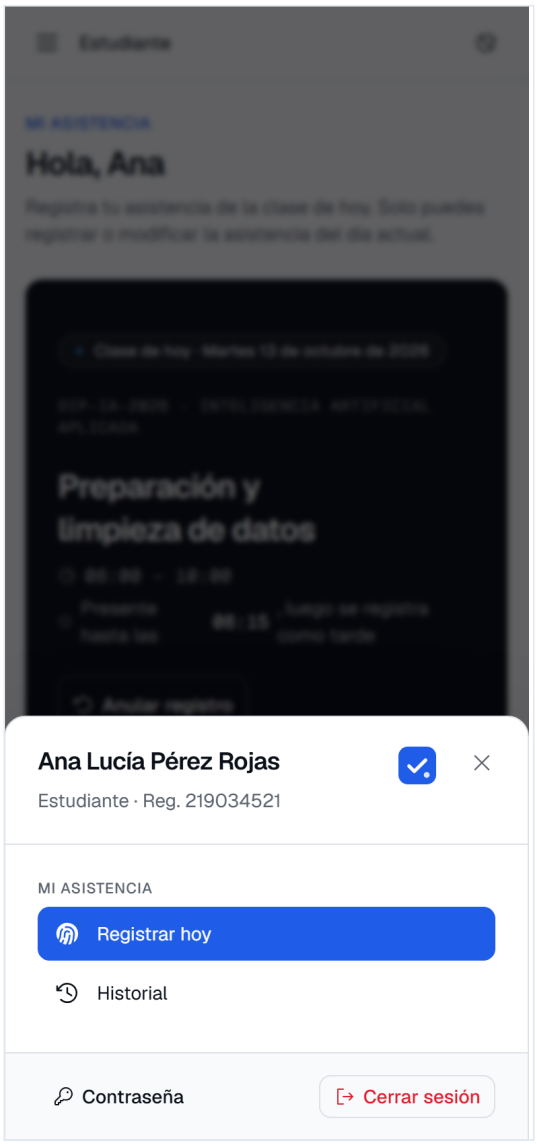


Figura 22. Menú móvil en modal.

6 Referencia de la API

Prefijo `/api/v1`. La documentación interactiva está en `http://localhost:8000/docs`. Todas las rutas, salvo el login, requieren el encabezado `Authorization: Bearer <token>`.

Método	Ruta	Rol	Descripción
GET	/health	Público	Estado del servicio
POST	/auth/login	Todos	Iniciar sesión
GET	/auth/me	Todos	Usuario autenticado
POST	/auth/change-password	Todos	Cambiar contraseña
GET	/courses	Todos	Listar programas
POST	/courses	Administrador	Crear programa
PATCH	/courses/{course_id}	Administrador	Editar programa
GET	/courses/{course_id}/sessions	Administrador	Sesiones (horarios) de un programa
POST	/courses/{course_id}/sessions	Administrador	Crear sesión manualmente
PATCH	/sessions/{session_id}	Administrador	Editar sesión
DELETE	/sessions/{session_id}	Administrador	Eliminar sesión
GET	/students	Administrador	Listar estudiantes con resumen
POST	/students	Administrador	Registrar estudiante o inscribir uno existente en un programa
GET	/students/{student_id}	Administrador	Detalle de estudiante
PATCH	/students/{student_id}	Administrador	Editar estudiante
GET	/students/{student_id}/attendance	Administrador	Historial de asistencia de un estudiante
GET	/attendance/sessions/{session_id}	Administrador	Matriz de asistencia de una sesión
PUT	/attendance/sessions/{session_id}/students/{student_id}	Administrador	Registrar o modificar asistencia (admin, cualquier fecha)
GET	/me/attendance	Estudiante	Mi asistencia (sesión de hoy, historial y estadísticas)
POST	/me/attendance/{session_id}	Estudiante	Registrar mi asistencia (solo el día de hoy)
DELETE	/me/attendance/{session_id}	Estudiante	Anular mi registro de hoy
POST	/imports/students	Administrador	Importar estudiantes
POST	/imports/sessions	Administrador	Importar horarios y fechas
GET	/imports	Administrador	Historial de importaciones
GET	/imports/templates/{kind}	Administrador	Descargar plantilla Excel
GET	/dashboard/stats	Administrador	Indicadores del panel
GET	/reports/attendance.xlsx	Administrador	Exportar asistencia a Excel

7 Modelos de importación Excel

Cada importación ofrece un **modelo descargable** con encabezados, filas de ejemplo, formatos de celda y una hoja de instrucciones. Los encabezados aceptan sinónimos comunes (por ejemplo «correo» por «email» o «programa» por «carrera»).

7.1 Estudiantes · modelo_estudiantes.xlsx

Columna	Obligatoria	Contenido
`registro`	Sí	Número de registro único; es el usuario de ingreso.
`nombre`	Sí	Nombre completo.
`email`	No	Correo institucional válido.
`carrera`	No	Carrera o especialidad.

7.2 Horarios · modelo_horarios.xlsx

Columna	Obligatoria	Contenido
`fecha`	Sí	AAAA-MM-DD, DD/MM/AAAA o celda de fecha.
`hora_inicio`	Sí	HH:MM en formato de 24 horas o celda de hora.
`hora_fin`	Sí	Posterior a la hora de inicio.
`tema`	No	Tema o contenido de la sesión.

Validación fila por fila
Las filas con errores (registro vacío, correo inválido, fecha o rango horario incorrecto, duplicados) se rechazan con su número de fila y el motivo; el resto se importa. Cada lote queda en el historial de importaciones.

8 Instalación y ejecución

8.1 Con Docker (recomendado)

Levanta PostgreSQL, la API (aplica migraciones y carga datos de demostración) y el frontend:

```
docker compose up --build
```

- Frontend: `http://localhost:3001`
- API y documentación OpenAPI: `http://localhost:8000/docs`

8.2 Desarrollo local

```
docker compose up -d db
cd backend
uv sync
uv run alembic upgrade head
uv run python -m app.seed --demo
uv run uvicorn app.main:app --reload

cd frontend
pnpm install
pnpm dev --port 3001
```

8.3 Variables de entorno principales

Variable	Valor por defecto	Uso
`DATABASE_URL`	`postgresql+psycopg://…:5433/asistencia`	Conexión a PostgreSQL.
`JWT_SECRET`	`(cambiar en producción)`	Firma de los tokens.
`DEFAULT_PASSWORD`	`admin1234!`	Contraseña común de las cuentas nuevas.
`ADMIN_REGISTRATION`	`218166141`	Registro del administrador inicial.
`LATE_TOLERANCE_MINUTES`	`15`	Minutos de tolerancia antes de «tarde».
`TIMEZONE`	`America/La_Paz`	Zona horaria oficial del sistema.
`API_URL (frontend)`	`http://localhost:8000`	URL interna de la API para el BFF.

8.4 Calidad

- 25 pruebas con pytest sobre autenticación, reglas de asistencia, multiprograma, importación y reportes: `uv run pytest`
- Análisis estático: `uv run ruff check` y `pnpm lint`
- Integración continua en GitHub Actions para backend y frontend.